

DOSSIER ACTIVIDADES - INTRODUCCIÓN A PHP

Instrucciones para la entrega del Dossier

Es **obligatorio** seguir la siguiente estructura y nomenclatura al subir los archivos de las actividades.

1. Estructura de Carpetas y nomenclatura de Archivos.

Cada uno debe crear una carpeta dentro de su rama de repositorio con la siguiente nomenclatura exacta, (respetando mayúsculas y guiones bajos) y subir (hacer COMMIT) los ficheros de las actividades con la nomenclatura exacta indicada:

- **Ruta de entrega:** [RAÍZ_RAMA_ESTUDIANTE]/Dossier_Actividades_Introduccion_PHP/

Ejemplo de estructura final esperada:

```
/tu_rama_personal/  
└─ ...  
└─ Dossier_Actividades_Introduccion_PHP/  
    ├── actividad_1_arrays.php  
    ├── actividad_2_cadenas.php  
    ├── actividad_3_variables.php  
    ├── actividad_4_funciones.php  
    ├── actividad_5_excepciones.php  
    └─ actividad_6_clases_objetos.php
```

2. Entrega del dossier.

Además de subir los ficheros .php, hay que entregar el dossier en formato PDF en la actividad asociada en AULES.

Para cada una de las 6 actividades, debes rellenar la tabla asociada en el documento con los siguientes elementos:

1. **Código del Ejercicio:** Copia y pega el código PHP completo (.php) que has desarrollado para solucionar la actividad.
2. **Captura de Pantalla de la Salida:** Ejecuta el fichero .php en tu terminal o navegador y adjunta una captura de pantalla (pantallazo) que demuestre la salida obtenida por el script. Esto valida que el código es funcional.

1. Arrays: Manipulación, Ordenación y Verificación

Las **funciones de arrays** son cruciales en PHP para gestionar colecciones de datos.

- `ksort()`: Ordena un array asociativo por sus **claves** en orden ascendente.
- `sort()`: Ordena un array por sus **valores** en orden ascendente (reindexa el array numéricamente).
- `array_values()`: Devuelve todos los valores del array en un nuevo array indexado numéricamente.
- `array_keys()`: Devuelve todas las claves del array.
- `array_key_exists()`: Comprueba si una clave o índice existe en un array.

Tareas: Crea un array asociativo llamado \$países_capitales con 5 países y sus capitales.

1. **Ordena** el array alfabéticamente por el **nombre del país** (la clave) e imprime el resultado.
2. **Ordena** el array alfabéticamente por el **nombre de la capital** (el valor) e imprime el resultado.
3. Crea un nuevo array llamado \$capitales que contenga solo los **valores** (las capitales) del array original e imprímelo.
4. Crea un nuevo array llamado \$países_claves que contenga solo las **claves** (los países) del array original e imprímelo.
5. Verifica si la clave "Francia" existe en \$países_capitales y muestra un mensaje indicando si existe o no.

CÓDIGO FUENTE	SALIDA

2. Cadenas (Strings): Longitud y Transformación

Las **funciones de cadenas** son esenciales para manipular texto.

- strlen(): Devuelve la **longitud** de una cadena (el número de bytes).
- strtolower(): Convierte una cadena a **minúsculas**.

Tareas: Define una cadena llamada \$frase con el valor "eSTe eS Un TeXto DE PrUEBa cON MÚTipleS MaYúSCULAS."

1. Calcula y muestra la **longitud** de la cadena \$frase.
2. Convierte la cadena \$frase a **minúsculas** y muestra el resultado.
3. Define una segunda cadena llamada \$palabra_corta con el valor "PHP" y calcula su longitud.
4. Crea una nueva cadena que sea la concatenación de la cadena en minúsculas del punto 2 y la longitud del punto 3. Muestra el resultado.

CÓDIGO FUENTE	SALIDA

3. Variables: Verificación de Estado

Las **funciones de variables** permiten determinar el estado de una variable.

- isset(): Determina si una variable **está definida** y no es NULL. Devuelve false si la variable no existe o es NULL.
- empty(): Determina si una variable **está vacía**. Se considera vacía si no existe o su valor es equivalente a false (ej: "", 0, NULL, false, array()).
- is_null(): Comprueba si una variable tiene el valor **NULL**.

Tareas: Define las siguientes variables:

```
$nombre = "Juan";  
$edad = 0;  
$saldo; // No definida  
$email = null;  
$lista = array();
```

(Asume que \$saldo solo se declara sin asignación inicial en el entorno de pruebas, lo que para isset() la trataría como no definida).

Para cada una de las siguientes variables (\$nombre, \$edad, \$saldo, \$email, \$lista), aplica y muestra el resultado de:

- 1. `isset()`
- 2. `empty()`
- 3. `is_null()`

Crea una tabla en el resultado para organizar la información.

CÓDIGO FUENTE	SALIDA

4. Funciones Definidas por el Usuario: Paso de Parámetros

Al definir funciones, podemos pasar parámetros de dos formas principales:

- **Paso por valor (Copia):** Se pasa una **copia** del valor de la variable a la función. Cualquier cambio dentro de la función **no afecta** a la variable original fuera de ella.
- **Paso por referencia:** Se pasa una **referencia** (un "alias") de la variable original a la función, utilizando el símbolo **&** delante del parámetro en la definición de la función. Cualquier cambio dentro de la función **afecta** a la variable original.

Tareas:

- 1. Define una función llamada `incrementar_copia($numero)` que incremente el valor del parámetro en 1 y lo muestre.
- 2. Define una variable `$contador = 10`. Llama a la función `incrementar_copia` pasándole `$contador`. Después de la llamada, muestra el valor de `$contador` para verificar si ha cambiado.
- 3. Define una función llamada `incrementar_referencia(&$numero)` que incremente el valor del parámetro en 1 y lo muestre. **Nota el &.**
- 4. Define una variable `$puntos = 5`. Llama a la función `incrementar_referencia` pasándole `$puntos`. Después de la llamada, muestra el valor de `$puntos` para verificar si ha cambiado.

CÓDIGO FUENTE	SALIDA
---------------	--------

--	--

5. Excepciones y Errores: Manejo Básico

En PHP, el manejo de errores y excepciones permite gestionar situaciones inesperadas o errores lógicos sin detener bruscamente la ejecución del programa.

- **Excepciones:** Se utilizan para errores lógicos o de programación que el programa puede esperar y manejar. Se controlan con el bloque try...catch.
- **try:** Contiene el código que podría lanzar una excepción.
- **catch:** Captura la excepción lanzada y define la lógica de manejo.
- **Errores (en versiones modernas de PHP):** Muchos errores fatales o de sintaxis que antes no se podían capturar ahora implementan la interfaz Throwable y pueden ser capturados.

Tareas:

1. Define una función llamada dividir(\$numerador, \$denominador) que intente realizar una división.
2. Dentro de la función, utiliza un bloque if para comprobar si el \$denominador es **cero**. Si lo es, lanza una **nueva excepción** con el mensaje: "Error: División por cero no permitida.". Si no es cero, devuelve el resultado de la división.
3. Utiliza un bloque try...catch para llamar a la función dividir() en dos escenarios:
 - Una división válida: dividir(10, 2)
 - Una división inválida: dividir(5, 0)
4. En el bloque catch, muestra el mensaje de la excepción capturada (getMessage()).

CÓDIGO FUENTE	SALIDA

6. Clases y Objetos: Propiedades Estáticas, Constructor y __toString()

Las clases son la base de la Programación Orientada a Objetos (POO) en PHP.

- **Propiedades Estáticas (static):** Pertenecen a la **clase** en sí, no a una instancia (objeto) específica. Se acceden usando el operador de resolución de ámbito doble dos puntos (::), por ejemplo, NombreClase::\$propiedadEstatica.
- **Constructor (__construct):** Es un método especial que se ejecuta automáticamente cuando se **crea un nuevo objeto** de la clase (usando new). Se utiliza para inicializar las propiedades del objeto.

- **Método Mágico __toString():** Es un método público que, si se define, permite a un objeto ser tratado como una **cadena (string)**. PHP lo llama automáticamente cuando intentas imprimir el objeto (por ejemplo, con echo).

Tareas: Crea una clase llamada Producto que represente un artículo en un inventario.

1. Define una **propiedad estática** privada llamada contadorProductos e inicialízala a **0**.
2. Define dos propiedades públicas de instancia: nombre y precio.
3. Define el **constructor** de la clase (__construct). Este debe:
 - Recibir los parámetros \$nombre y \$precio.
 - Asignar los valores a las propiedades del objeto (\$this->nombre, \$this->precio).
 - **Incrementar** el valor de la propiedad estática contadorProductos para registrar el nuevo producto.
4. Define un método **público estático** llamado obtenerTotalProductos() que devuelva el valor de contadorProductos.
5. Define el **método mágico** __toString() que devuelva una cadena con el siguiente formato: "Producto: [nombre] - Precio: [precio]€".
6. **Instancia** tres objetos de la clase Producto:
 - \$p1 = "Laptop", 1200.50
 - \$p2 = "Ratón", 25.99
 - \$p3 = "Teclado", 75.00
7. Muestra en pantalla la información de los objetos \$p1 y \$p3 usando echo (esto probará la función __toString()).
8. Muestra el número total de productos instanciados utilizando la función estática obtenerTotalProductos().

CÓDIGO FUENTE	SALIDA